

EPIC 08 Task 03 - Dockerize Microservices

Your task says:

Create Dockerfiles for all services.

Create docker-compose.yml and run:

- Gateway
- Services
- Database
- Redis
- Kafka

I'll give you the implementation step-by-step.

Step 1 — Create a Dockerfile for each service

For example, inside your **Order Service** project, create a file named:

Dockerfile

Put this inside:

```
FROM eclipse-temurin:17-jdk-alpine
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8081
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Do the same for your other microservices.

For example:

gateway-service/

Dockerfile

order-service/
Dockerfile

payment-service/
Dockerfile

inventory-service/
Dockerfile

Change the EXPOSE port for each service to its actual port.

Step 2 — Build the JAR files

Open a terminal in each Spring Boot project.

Run:

```
mvn clean package -DskipTests
```

After successful completion, you should have:

target/
your-service-name.jar

For example:

```
order-service/  
├─ src/  
├─ pom.xml  
├─ Dockerfile  
└─ target/  
    └─ order-service.jar
```

Step 3 — Create docker-compose.yml

Create one file at the **root folder** containing all your services:

```
microservices-project/
|
|— gateway-service/
|   └─ Dockerfile
|
|— order-service/
|   └─ Dockerfile
|
|— payment-service/
|   └─ Dockerfile
|
|— inventory-service/
|   └─ Dockerfile
|
└─ docker-compose.yml
```

Put this in docker-compose.yml:

services:

gateway:

build: ./gateway-service

container_name: gateway-service

ports:

- "8080:8080"

depends_on:

- order-service

- payment-service

order-service:

build: ./order-service

container_name: order-service

ports:

- "8081:8081"

depends_on:

- postgres

- redis

payment-service:
build: ./payment-service
container_name: payment-service
ports:
- "8082:8082"
depends_on:
- postgres

inventory-service:
build: ./inventory-service
container_name: inventory-service
ports:
- "8083:8083"
depends_on:
- postgres
- kafka

postgres:
image: postgres:16
container_name: postgres
environment:
POSTGRES_DB: microservices
POSTGRES_USER: postgres
POSTGRES_PASSWORD: postgres
ports:
- "5432:5432"
volumes:
- postgres-data:/var/lib/postgresql/data

redis:
image: redis:7
container_name: redis
ports:
- "6379:6379"

kafka:
image: bitnami/kafka:3.7
container_name: kafka

ports:

- "9092:9092"

environment:

KAFKA_CFG_NODE_ID: 1

KAFKA_CFG_PROCESS_ROLES: broker,controller

KAFKA_CFG_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093

KAFKA_CFG_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093

KAFKA_CFG_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092

KAFKA_CFG_CONTROLLER_LISTENER_NAMES: CONTROLLER

KAFKA_CFG_AUTO_CREATE_TOPICS_ENABLE: "true"

volumes:

postgres-data:

Important: If your actual service names or ports are different, use those names in the Compose file.

Step 4 — Change database configuration

Inside each service's application.properties, don't use localhost for containers.

For example:

spring.datasource.url=jdbc:postgresql://postgres:5432/microservices

spring.datasource.username=postgres

spring.datasource.password=postgres

For Redis:

spring.data.redis.host=redis

spring.data.redis.port=6379

For Kafka:

spring.kafka.bootstrap-servers=kafka:9092

Here:

postgres
redis
kafka

are the Docker Compose service names.

Step 5 — Start everything

Open a terminal in the folder containing:

docker-compose.yml

Run:

docker compose up --build

Docker will build and start:

Gateway

↓

Order Service

Payment Service

Inventory Service

↓

PostgreSQL

Redis

Kafka

Step 6 — Check running containers

Open another terminal and run:

docker compose ps

You should see the containers running.

You can also use:

`docker ps`

Step 7 — Stop the application

When finished:

`docker compose down`

To remove the database volume as well:

`docker compose down -v`

Task 03 completed

You have implemented:

- ☒ Dockerfile for each microservice
- ☒ docker-compose.yml
- ☒ Gateway
- ☒ Microservices
- ☒ PostgreSQL database
- ☒ Redis
- ☒ Kafka
- ☒ Docker networking and service communication